

PALS: A High-Speed ML-DSA Hardware Accelerator with Enhanced Pipeline on FPGA

Pengfei Sun¹, Qirui Wang¹, Makoto Ikeda¹

¹ The University of Tokyo, Department of Electrical Engineering

Abstract—This paper presents PALS, a high-speed and resource-efficient ML-DSA accelerator with a fully pipelined architecture, supporting all security levels. Implementation on Artix-7 and UltraScale+ platforms shows that, compared with the best previous work, our design reduces clock cycles, latency, and Area-Time Product (ATP) by approximately 40%, 25%, and 10%, respectively.

Index Terms—Field Programmable Gate Array (FPGA), hardware accelerator, Module-Lattice-Based Digital Signature Algorithm (ML-DSA), Post-Quantum Cryptography (PQC).

I. INTRODUCTION

As quantum computing evolves, the ML-DSA digital signature has been standardized by the National Institute of Standards and Technology [1]. To meet its high computational demands, researchers have proposed high-performance hardware accelerators for ML-DSA [2]–[4]. However, a major bottleneck limiting top-level efficiency in existing designs is the data formats among different modules are incompatible, thus hindering efficient co-operation and forcing pipelines to be interrupted [2], [3].

In this paper, we propose PALS extending our preliminary research in [5], which addressed only the reordering unit, by integrating it into a fully pipelined architecture with speed-matching modules and supporting all ML-DSA security levels. We solve the above challenges with the following innovations:

- 1) The first fully pipelined collaborative architecture for the ML-DSA accelerator: By integrating a further optimized reordering mechanism, PALS allows all calculation modules to be connected in pipeline and reduces the total clock cycles of the most time-consuming reordering related operations by approximately 37%.
- 2) A speed-matching submodule design: PALS achieves a high-throughput pipeline by engineering all submodules to operate in a speed-matched manner. This approach allows savings achieved by designing pipelines in specific operations to be fully utilized.

On Artix-7 and UltraScale+ platforms, compared with the state-of-the-art ML-DSA accelerators, PALS reduces the total clock cycles by approximately 40% and reduces ATP by 10%, demonstrating its efficiency across all ML-DSA security levels.

II. OPTIMIZATION PROPOSAL

Fig. 1 illustrates the FSM (finite state machine) transition logic during Key Generation (a function defined in ML-DSA) in PALS. The X-axis denotes states, and the Y-axis represents dataflow directions between modules, which will be introduced in Section III Architecture. For example, in Key Generation State 3, which is the most time-consuming part of Key Generation, our work provides pipelined connection through all submodules, which is due to the pipeline reordering in PALS. In contrast, conventional approaches require two separate processing stages: 1) computing $A*s_1$, 2) storing/retrieving intermediate results via BRAM before

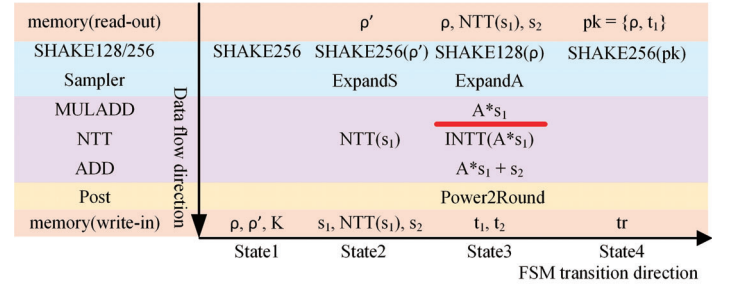


Fig. 1: FSM Design in Key Generation, redline marks the breakpoint in previous works [2]–[4].

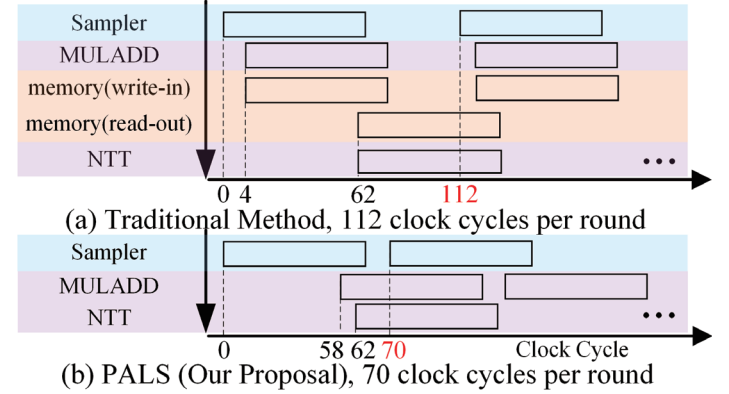


Fig. 2: Comparison of Clock Cycles in Key Generation State 3, PALS eliminates RAM buffering, reducing delay by 42 cycles.

executing INTT (inverse Number Theoretic Transform), as the dividing red line in Fig. 1.

Fig. 2 compares the Key Generation process between (a) traditional method and (b) our proposal. The traditional approach requires a delay of at least 62 clock cycles to write data from Sampler to BRAM before it can be read in the format required by INTT. This procedure would occupy the BRAM resources and delay the next sampling round. In contrast, our solution enables SHAKE128/256 and Sampler to almost immediately start pipeline reordering after completing an output, without waiting. This innovation allows PALS to begin the calculation of the second polynomial 42 cycles earlier, resulting in an approximately 37% reduction in clock cycles. For Key Generation under Security Level 5, Key Generation State 3 takes up 78% time. Through our optimized design, we achieved a clock cycle reduction of approximately 28% compared to the pipeline-severing design [2].

III. ARCHITECTURE

The high-level architecture of PALS shown in Fig. 3 is divided into two parts: *Memory* and *Processing Unit*.

The *Memory* stores polynomials in *Polynomial RAM* and the seeds of hashing in *SHAKE RAM*. The *Polynomial RAM* is instantiated as three 92-bit wide dual-port RAMs, enabling simultaneous read or write of 4 polynomial coefficients. The *SHAKE RAM* is instantiated as a 64-bit wide simple-dual-port RAM. A *Controller* controls FSM according to whether the RAM addresses reach a specific location, thereby directing the *Processing Unit* to execute ML-DSA defined functions.

The *Processing Unit* is responsible for computations in the ML-DSA. PALS introduces a Polynomial Multiplication and

TABLE I. Implementation Results and Comparison

	Security Level	Cycles ¹ ($\times 10^3$)	Freq. (MHz)	Latency ¹ (μ s)	LUTs	FFs	BRAMs	DSPs	Area ² ($\times 10^3$)	ATP ³ ($\times 10^6$)
[2] Artix-7	2	4.2/31.6/4.4	96.9	43.3/326/45.4	29998	10366	11	10	12.0	5.0
	3	5.9/49.5/6.1		60.9/511/63.0						7.6
	5	8.8/55.3/9.0		90.1/571/92.9						9.0
[3] Artix-7	2	4.4/30.1/5.0	130	33.9/231/38.5	32320	9734	14	24	14.5	4.4
	3	7.3/49.5/8.0		56.1/381/61.6						7.2
	5	10.9/55.2/12.7		83.8/425/97.6						8.8
[4] Artix-7	2	5.3/33.5/4.9	114.6	46.2/292.3/42.8	20621	8104	16	8	10.2	3.9
	3	7.8/53.6/7.6	109	71.6/491.7/69.7	23378	9079	20		11.8	7.5
	5	12.5/72.2/11.8	114.3	109.4/631.7/103.2	28791	11338	24		14.2	12.0
This Work Artix-7	2	2.6/16.2/3.4	103	24.9/157.2/33.3	33799	14033	16	40	17.3	3.7
	3	4.1/27.0/5.0		39.7/262.5/49.0						6.1
	5	6.5/32.1/7.8		63.2/312.0/75.5						7.8
[3] UltraScale+	2	4.4/30.1/5.0	300	14.7/100/16.7	32035	9707	14	24	14.4	1.9
	3	7.3/49.5/8.0		24.3/165/26.7						3.1
	5	10.9/55.2/12.7		36.3/184/42.3						3.8
This Work UltraScale+	2	2.6/16.2/3.4	251	10.2/64.5/13.7	34536	13952	16	40	17.6	1.6
	3	4.1/27.0/5.0		16.3/107.7/20.1						2.5
	5	6.5/32.1/7.8		25.9/128.0/31.0						3.3

¹ The three numbers in the Clock and Latency columns represent the average results of 50 tests of Key Generation, Signing, and Verifying defined by ML-DSA [1]

² $Area = LUT/4 + FF/8 + BRAM \times 200 + DSP \times 100$ [6] ³ ATP: Area-Time Product, $ATP = Area \times \sum Latency$

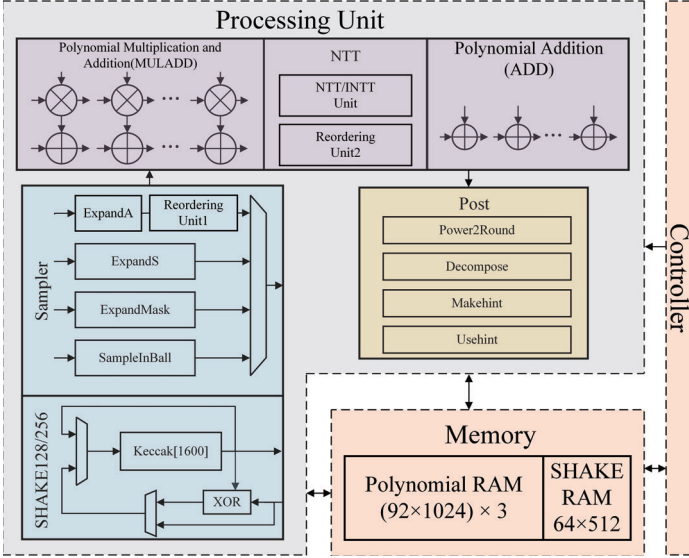


Fig. 3: High-Level Architecture.

Addition (*MULADD*) upstream of *NTT*, and a Polynomial Addition (*ADD*) downstream of *NTT* to manage coefficient-wise multiplication/addition/subtraction between polynomial matrices and vectors operations. The additional modules enable concurrent execution of *NTT*-domain computations alongside *NTT/INTT* operations, thereby supporting our pipeline and FSM design. *SHAKE128/256* and *Sampler* complete the hashing and sampling operations defined in [1]. *Reordering Unit1&2* are designed to support pipelined data transfer between modules without the need for writing to *Memory*. PALS optimizes resource consumption by replacing the 1800+ registers in the *Reordering Unit1* from [5] with a simple 92x128 dual-port RAM, with a carefully designed address transition logic.

IV. IMPLEMENTATION RESULTS AND COMPARISON

PALS is implemented on FPGA Artix-7 and Virtex UltraScale+. Table I shows the post place-and-route performance and

resource consumption of PALS to compare with related works on the same platforms. The three numbers in the Cycles and Latency columns represent the average results for Key Generation, Signing, and Verifying. Area and Time are computed using a commonly used formula [6] and the sum of the three functions' latencies, respectively, with ATP being their product.

Although some submodules designed to support our FSM and pipeline result in a larger Area, PALS achieves the best Cycles and Latency results, saving around 40% and 25%, respectively. Our FSM and pipeline designs bring the effect introduced in Section II and address the data reordering bottleneck in previous works [2], [3] by concealing clock cycles and enabling concurrent *NTT* and polynomial operations. The effect is especially significant in Signing due to repeatedly call. This approach also ensures efficient resource sharing across all security levels, unlike some designs that only pipeline within modules [4]. As a result, our implementation achieves a significant reduction in latency, ultimately outperforming the best previous work in ATP by approximately 10% reduction.

REFERENCES

- [1] NIST, "Module-Lattice-Based Digital Signature Standard," 2024. [Online]. Available: <https://csrc.nist.gov/pubs/fips/204/final>
- [2] C. Zhao *et al.*, "A compact and high-performance hardware architecture for CRYSTALS-Dilithium," *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2022, no. 1, pp. 270–295, 2022, doi: 10.46586/tches.v2022.i1.270-295.
- [3] X. Li *et al.*, "A high speed post-quantum crypto-processor for Crystals-Dilithium," *IEEE Trans. Circuits Syst. II, Express Briefs*, vol. 71, no. 1, pp. 435–439, Jan. 2024, doi: 10.1109/TCSII.2023.3304416.
- [4] Z. Wu *et al.*, "An efficient hardware implementation of Crystal-Dilithium on FPGA," in *Information Security and Privacy-ACISP 2024*, Singapore, 2024, pp. 64–83, doi: 10.1007/978-981-97-5028-3_4.
- [5] P. Sun *et al.*, "Fully pipelined number theoretic transform and reordering solution for CRYSTALS-Dilithium," in *Proc. IEEE Int. Conf. Integr. Circuits, Technol. Appl. (ICTA)*, Hangzhou, China, 2024, pp. 100–101, doi: 10.1109/ICTA64028.2024.10860673.
- [6] M. Li *et al.*, "Reconfigurable and high-efficiency polynomial multiplication accelerator for CRYSTALS-Kyber," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 42, no. 8, pp. 2540–2551, Aug. 2023, doi: 10.1109/TCAD.2022.3230359.